Statement of Career Goals

Christian Kästner, 2026

I am trying to keep this short: I am a software engineering educator and researcher with broad interests. I have worked on diverse topics within software engineering with many collaborators, sometimes bordering other disciplines such as machine learning, programming languages, and human-computer interaction, but my home is the software engineering community. My work spans software variability and reuse, developer collaboration, open-source sustainability, software supply chain security, and software engineering for and with AI. While I do not actively pursue this as an explicit agenda, almost all my work directly or indirectly seeks ways of dealing with *imperfect modularity*, as I will explain. Methodologically, I lean on empirical methods and pragmatic tool building.

On the education side, I aim to prepare future software engineers for the complexities of the real world and how to approach them responsibly. I have taught and substantially revised several software engineering courses at CMU. Since 2019, my focus has been software engineering education for data scientists and ML engineers, through the course (and book) *ML in Production*, now taught at CMU every semester to 80-150 students.[1] In this course, I look at how software engineering needs to adapt when building products with ML components, how it can be taught to data scientists and others with little software engineering exposure, and now also how things shift again when much of the software engineering work itself gets automated by AI tooling.

Regarding service, I focus on high-impact activities around supporting students, hiring, peer review, and community building.

Research Contributions

My broad goal in research is to help software engineers build complex systems in the real world and to do so responsibly. More specifically there are three main areas of research that my group and I have conducted at CMU: Scaling variability and reuse, understanding open source ecosystems, and responsible engineering of ML-powered applications.

Scaling variability and reuse. Variability and reuse are how complex modern software scales: A single configurable codebase like the Linux kernel or a modern web framework can serve millions

[1] https://mlip-cmu.github.io

of distinct use cases, and systematic reuse is what makes large software systems economically feasible to build and maintain. But this leverage comes with a cost when thousands of interacting options create combinatorial spaces that are hard to reason over manually and resist traditional testing and automated analyses, leading to defects that hide in specific configurations.

Helping developers manage complexity from variation and reuse has been the central focus of much of my earlier research. I championed pragmatic variability management with simple tools (similar to, but more disciplined than the C preprocessor), rather than trying to create new language constructs, which was the prevailing trend at the time. My students, my collaborators, and I showed how to scale analyses across exponential configuration spaces, such as type checking all 2^{14k} compile-time configurations of the Linux kernel, using SAT solvers (Kästner et al. 2011; Rhein et al. 2018), or how to help developers test and debug in complex configurable systems (Velez et al. 2022; Medeiros et al. 2016; Lillack et al. 2017). With these approaches, we found tricky problems that only emerge when multiple options interact. Our work spanned multiple variability mechanisms, from the C preprocessor to feature flags, and different qualities of interests, from type safety to performance. We realized that most of these analyses were actually feasible at scale despite exponential combinatorics, because essential configuration complexity is much lower in practice than what is theoretically possible (Meinicke et al. 2016; Liebig et al. 2011), likely because developers write code in ways they can still mentally manage. The tools we developed, like CIDE and TypeChef, and corresponding evaluation harnesses, were adapted by many others in the research community, and our work triggered a resurgence in the interest in principled alternatives to the C preprocessor. The PhD theses from my advisees Chu-Pan Wong, Miguel Velez, and Jens Meinicke,[2] as well as the post-doc work of Pooyan Jamshidi were all part of this multi-year effort, funded from several NSF grants, including a Career award. Early parts of this work received the 10-year Most Influential Paper Award at the International Conference on Software Product Lines in 2019.

Understanding open source ecosystems. Open source is the critical infrastructure underlying nearly all modern software (Eghbal 2020), a public good built often by volunteers with no formal obligation to the downstream users who depend on their code. Supply-chain attacks, maintainer retirement and burnout, and complex dependency and fork networks are now straining trust. Low trust communities are inefficient (Putnam 2015; Mayer et al. 1995), and we now see signs of this in open source with rising auditing and rework costs, and threats to distributed coordination and collaboration.

For the last decade, my research has sought to better understand how these communities work, with a focus on collaboration, sustainability, and security, and to build interventions that help these communities adapt and continue serving the developers and society that depend on them. Three of my advisee's dissertations capture the core of this agenda: In Shurui Zhou's dissertation

[2] Jens Meinicke formally was in a PhD program at my alma mater University of Magdeburg, but was effectively acting as a member of my research group and spent several years as a visiting scholar at CMU.

work, we explored how developers coordinate across forks and how to improve efficiency in that process (Zhou et al. 2018, 2019). In Courtney Miller's dissertation work, we explored why maintainers leave open source and how this impacts others, and how those impacts can be lessened (Miller et al. 2019, 2025). And in Hao He's dissertation work we explored open-source security controversies like whether pinning or floating is more secure or whether fake stars undermine trust in open-source software supply chains (He et al. 2025, 2026). This work is often empirical, but we also build technical interventions, such as lightweight sandboxes for dependencies or notification tools for developers for environments in which traditional trust signals erode (Ferreira et al. 2021; Miller et al. 2026). We shared our results with practitioner communities through talks at practitioner conferences and through podcasts, including JSConf and the Sustain podcast. The security work has been supported by a large, multi-site NSF Frontier award, establishing the Secure Software Supply Chain Center.[3] Three of the resulting papers have received distinguished paper awards at ICSE and FSE (He et al. 2025; Miller et al. 2025, 2022).

Responsibly engineering ML-powered applications. Software systems increasingly rely on machine-learning components for core functionality, opening up capabilities that were previously out of reach, but also breaking traditional software engineering assumptions about specifications, testing, and modularity. ML components are fundamentally unreliable, yet the engineering needed to build dependable systems *around* them is rarely taken seriously in practice. ML-powered projects reportedly fail far more often than traditional ones, often rushed by teams without strong software engineering backgrounds, amid public discourse heavy on hype and light on rigor. Further, AI coding agents are now disrupting how software itself is built, raising the urgency further.

Since 2019, much of my research and teaching has shifted to how to responsibly build software with ML components – both to bring software engineering rigor to this emerging field and to prepare students and developers for a future where they will routinely work with unreliable ML components and the risks that come with them. Central here is Nadia Nahar's dissertation work revealing that many practical problems emerge exactly at the interdisciplinary boundary between software engineers and data scientists; we developed several strategies to improve mutual understanding and collaboration with tools that anticipate misconceptions and document assumptions, with pedagogical interventions, with policy design, and with approaches to anticipate and mitigate mistakes (Nahar, Yang, et al. 2026; Nahar, Omar, et al. 2026; Nahar et al. 2022; Hong et al. 2025; Bhat et al. 2023; Nahar et al. 2024). In addition, Chanyang Yang's dissertation work explored how to adapt traditional software assurance methods for data science code and how to ground ML model testing in requirements engineering practices (Yang, Rustogi, et al. 2023; Yang, Brower-Sinning, et al. 2023; Yang et al. 2021, 2026). Much of this work was highly interdisciplinary, involving collaborators from sociology, medicine, machine learning, human-computer interaction, and security, and was published beyond software engineering

[3] https://s3c2.org

venues. Our focus on collaboration challenges and requirements-grounded testing resonated with practitioners and students. I wrote a textbook, published open access with MIT Press, that covers the entire software engineering process from an AI engineering lens far beyond our own work (Kästner 2025). Our papers received awards from ICSE, CAIN, and CHI (Nahar et al. 2022, 2023; Nahar, Yang, et al. 2026).

A unifying theme: Imperfect modularity. While I follow my evolving interests rather than a single grand agenda, most of my work gravitates around the shared theme of *imperfect modularity*. Modularity is a fundamental principle to manage complexity; it allows software engineers to scale system design, collaboration, quality assurance, and maintenance beyond what a single developer can handle. However, modularity requires assumptions that are often unrealistic in real systems (Ostermann et al. 2011), such as having stable requirements, clear interface specifications that capture all interactions, and opportunities to hide complex internals behind abstractions. For example, many practical problems emerge from interactions between behaviors of different modules in ways that are not captured in module interfaces, such as two modules competing for resources or undermining each other's assumptions or two ML models compensating for each other's mistakes in unpredictable ways – this is often described as *feature interactions* in software engineering, as *emergent properties* in systems engineering, or as the *changing anything changes everything* phenomenon in machine learning.

My variability work explored exactly the places where modularity breaks down, such as interactions among features often emerge at run time in ways that are difficult to predict by their modular interfaces alone, while adding more information to interfaces can undermine the benefit of information hiding – here pragmatic developer tooling can help where modularity falls short, for example, fostering transparency and providing navigation support in non-modular implementations (Kästner et al. 2008). In addition, many analyses I developed strongly benefit from how developers approach software design modularly, even if their abstractions are leaky and require non-modular analysis (Meinicke et al. 2016; Velez et al. 2022). My open-source work grapples with the same tension: supply chains look modular on the surface, but breaking changes (i.e., unanticipated and often inevitable changes to module interfaces), vulnerabilities, and abandonment ripple across them; supply chain attacks resist modular containment; and practical development work resists the careful preplanning required for modularly and happens dynamically across many modules, forks, and projects rather (Zhou et al. 2018; Bogart et al. 2016; He et al. 2025). And finally, though it took me quite some time to clearly recognize this, machine learning components are inherently anti-modular and it challenges how we traditionally build software: Because of how inductive learning works, ML models fundamentally lack the specifications that modular reasoning depends on (Apel et al. 2022; Kästner 2025). In resisting modularity, machine learning breaks many traditional assumptions in software engineering (that were often overly optimistic to begin with, even before ML), which requires approaches that account for real-world messiness when there are no clear and stable modular abstractions. This is at the core of what shapes my research and teaching in this field: A focus on such as grounding

model testing in system requirements, testing in production, flexible operations, a strong focus on safety engineering and risk management, and facilitating interdisciplinary collaboration that overcomes unrealistic assumptions held by the different disciplines involved (Yang, Rustogi, et al. 2023; Kästner 2025; Nahar et al. 2022; Hong et al. 2025).

I believe that modularity will remain an enduring core design principle that will allow us to manage complexity and scale systems, even in a future of increasingly powerful AI agents. But my work has taught me that modularity is an imperfect abstraction that does not survive contact with the real world (or with machine learning) without substantial concessions – whether it is in tooling to navigate beyond modular structure and cope with unplanned interactions (Kästner et al. 2008; Meinicke et al. 2016; Zhou et al. 2018; Ferreira et al. 2021), to help developers coordinate across modules and plan non-modular changes (Miller et al. 2026; Bogart et al. 2016; He et al. 2025; Bhat et al. 2023), to detect problems early and build modular protections around anti-modular ML models (Hong et al. 2026; Nahar, Yang, et al. 2026; Nahar et al. 2025), or to educate developers to recognize limitations and deliberately manage risks (Kästner 2025; Hong et al. 2025).

Education

My goal in teaching is to prepare students for the messy reality of building *software that matters* in the complexity of the real world, whether that is systems too large for any one person to understand, requirements that shift, incentives that pull toward engineering shortcuts, or decisions whose consequences extend well beyond the codebase, such as frustrating and undermining the dignity of users exposed to rushed and poorly designed software. I want to graduate students who will build software responsibly for a better society, which requires anticipating harms, managing risk, and producing work they can stand behind. Much of CS and ML education leaves students unprepared for this, retreating into clean abstractions or chasing benchmarks. I like teaching and I treat teaching itself as a design problem grounded in evidence. I follow where the need is, whether that means revising introductory courses or building a new offering as the world changes.

I believe I have had my largest impact through co-creating and teaching the course *Machine Learning in Production/AI Engineering* (17-645, taught or co-taught 11 times since Fall 2019, usually with enrollments of 80-150 students), and through the shared lecture materials and textbook that grew out of it (Kästner 2025).[4] I often describe the course informally as teaching software engineering principles to data scientists and product managers. It covers traditional software engineering topics like requirements, quality assurance, process, and DevOps, through the lens of how ML components change the way we build software. Because ML models are so clearly unreliable components, students more readily appreciate the need to plan for mistakes (safety

[4] In the interest of space, I am not going to discuss contributions to other courses I have taught and revised at CMU, including 17-214, 17-313, and 17-654.

engineering), to think carefully about process, to test the software system beyond the individual model, and to monitor the system in production. The remainder of this section illustrates how I approach teaching through several principled design choices in this course.

Interdisciplinary teamwork as a key learning goal. Building production ML systems requires interdisciplinary collaboration, but placing students in groups does not by itself teach them how to communicate and collaborate effectively, navigate disagreement, or hold each other accountable; like any complex skill, collaboration must be deliberately scaffolded, practiced, and supported with feedback (Lovett et al. 2023). I follow established evidence-backed pedagogical practices (Oakley et al. 2004; Stein and Hurd 2000) of instructor-assigned teams, deliberately combining students with contrasting ML and software engineering backgrounds for a semester-long project with multiple milestones. I give teams extensive flexibility, but require a team contract and peer evaluations focused on *team citizenship*. I dedicate a full lecture early in the course to teamwork in student teams, where among others, I prepare students for eventual problems with a *pre-mortem activity* in class (Veinott et al. 2010) and I explicitly practice handling conflict and having *difficult conversations* in another activity (Patton et al. 2021). I have developed a scalable mentoring infrastructure pairing each team with a TA as a mentor who debriefs on teamwork after every milestone. I train the TAs to encourage students to speak up about problems in their own teams and to moderate such discussions, building a safe space for students to experiment with strategies to address conflicts (Lovett et al. 2023). I designed many of these practices based on pedagogy literature and based on discussions with colleagues (both within the department and at Eberly center workshops).

Operating a system with environment interactions under production conditions. A central design goal of the team project is to push beyond common benchmark evaluations or hackathon-style prototypes toward end-user products that must withstand production conditions, where the environment matters and mistakes have stakes. For the central project, students build a movie recommendation service for a simulated Netflix-style streaming platform, extracting features from continuously growing log files rather than a static dataset, and deploying their service to serve up to 10 requests per second for most of the semester. For many ML-focused students, this is their first encounter with latency, scalability, and the infrastructure required to update and monitor a live service. Because the simulator models 1,000,000 users whose preferences evolve in response to recommendations, predictions have (possibly delayed) consequences, some with safety implications (e.g., recommending R rated movies to minors), many affecting operating costs. In addition, the simulated environment allows us to inject desirable difficulties (Bjork and Bjork 2011) such as data quality problems, load spikes, data poisoning attacks, and many other issues that do not matter when maximizing benchmarks but that they will likely face in real production systems.

Active learning and real-world cases at scale. To anchor abstract concepts in practice, nearly every lecture revolves around a commercial ML-powered product, such as automated clinical notes, sidewalk delivery robots, and recidivism risk tools. In each lecture, we conduct at least one

breakout activity in which students apply the course material to a practical case. To scale activities to 100+ students in a classroom, we approximate the standard *think-pair-share* format (Lyman 1981): Students discuss with neighbors, post their answers publicly to a Slack channel, and then debrief with the full class, which we facilitate by passing around a microphone. We also experiment with automated LLM-generated feedback on their answers in Slack. These discussions surface and help correct misconceptions students bring from prior experience or coursework (Lovett et al. 2023), and they expose a spectrum of defensible answers rather than a single right one. Student feedback suggests that many students appreciate these activities and attend lectures in person to participate. We offer to grade participation based on these activities, though we make participation grading optional (opt-in) to support autonomy and commitment (Cullen and Oppenheimer 2024).

Specifications grading with a safety net. I am a strong proponent of *specifications grading*, a contemporary alternative to traditional point-based grading in which students must meet a clear pass/fail specification for each deliverable (Nilson 2015). I have adopted it in all my courses since 2019. Clear pass/fail criteria aligned to learning objectives set high, transparent standards; a token-based retry system provides flexibility and a safety net for failure; the scheme also reduces grading effort and makes grading more reliable and predictable. This design provides a safe space for learning, provides flexibility and additional support where needed, while grading for (eventual) content mastery, cf. also "grading for equity" (Feldman 2024). In *ML in Production*, I provide a fully transparent grading scheme with every assignment and implemented a token system for flexibility and retries, allowing me to direct learning (Biggs 1996), set standards, and fix the grade boundaries at the beginning of the semester.

Conceptual grounding with applied practice. Software engineering education often has a reputation for being abstract and vague, either not "mathy" enough or not applied enough. Rather than escaping to formalism or focusing on teaching tools only, I still try to convey foundational concepts in lectures, such as requirements, tradeoff reasoning, safety thinking, and process – even if they are often social rather than formal. I emphasize and apply them through case studies to ground the material in reality, which is also the format I use for exams. At the same time, I match this with applied labs and projects, where students use state of the art tools like Apache Kafka, Weights & Biases, and Grafana. Student feedback frequently cites the tooling and project experience as the deciding factor in successful interviews for ML engineering internships and jobs.

Adapting to student AI use. The common reaction to student's AI use has been a retreat to in-person, high-stakes assessment, which throws out decades of evidence that feedback from frequent low-stakes assessment is what actually drives learning (Black and Wiliam 1998). AI use can certainly undermine learning when it short-circuits productive struggle (Bastani et al. 2025; Handa et al. 2025), but banning it fails to prepare students for the real world. We explicitly permit AI use and focus instead on shaping what students do with it. Because most of our students will not build software engineering tools, I avoid software engineering tasks as case studies or

assignments, and concentrate on end-user applications with a clear business case. As assignments solvable by current agents become trivial, we constantly adapt how we teach and assess learning and raise the floor, for example extending a mature 400 KLOC open source repository rather than a demo project; implementing hazard analysis to support a scalable analysis rather than performing it manually on a small example. We moved previously-written reflection questions to in-person checkpoints with a TA, and we find that team projects do useful work here too, since teammates hold each other accountable for understanding their own contributions. I watch the public discourse and talk with colleagues to continuously refine our approach

Shared teaching materials. I make course materials publicly available and support other instructors and self-learners using them. This includes sharing slides, readings, and assignments (and even an annotated bibliography of relevant academic resources), as well as privately sharing course infrastructure like the movie streaming simulator. I have released *lecture recordings* of two offerings on YouTube, many of which have hundreds of views (the first lecture has over 6000 views). In 2020, we published an article about the first offering of the course to motivate teaching this content (Kästner and Kang 2020) and later I wrote a textbook corresponding to the course, published with MIT Press in 2025 under a Creative Commons license (Kästner 2025).[5] I am aware of related courses adopting substantial portions of the material at McGill (Jin Guo), Toronto (Marsha Chechik), and Groningen (Daniel Feitosa); in addition, several universities have independently created related courses with a similar scope since, e.g., (Chenoweth and Linos 2025; Siegmund 2025; Costa 2022; Palomba et al. 2026).

Service

I am selective in accepting service obligations and focus on high-impact activities important to me, usually around supporting students, hiring, peer review, and community building.

Within CMU. I have been stewarding the software engineering PhD program as its program director since 2019. I have been a member of the ISR/S3D hiring committee pretty much every year since I joined (often both teaching track and tenure track, chaired 2017-2019). I was the chair for the S3D department head search committee 2023/24.

Beyond CMU. I regularly am a reviewer for software engineering conferences and have received several distinguished reviewer awards. I served as PC chair for GPCE 2013, ASE 2018,[6] and CAIN 2027, as associate editor for TOSEM (2019-2022), and as conference chair for ICSE 2022. In 2025, I

[5] Publicly available version at https://mlip-cmu.github.io/book/

[6] ASE 2018 was the first major software engineering conference to shift from in-person PC meetings to online only PC discussions to scale reviewing. Many changes we made, including dedicated discussion days, are copied in PC instructions of ASE, ICSE, and FSE to this day.

helped redesign the ICSE Shadow PC program (a professional development program to teach the next generation of reviewers) from scratch, and will again co-organize it in 2026.

I also prioritize several pipeline and mentoring activities to support the next generation of software engineering researchers at an early stage. For many years, I have run the annual Feature-Oriented Software Development Meeting, an informal gathering (no formal publication, just an abstract submission) for mostly junior researchers to provide presentation, feedback, and networking opportunities, which has had a positive impact on that community and started many collaborations. I am currently building a new series on the same model with the Beyond the Model Meeting.[7] I have co-organized the ICSE Student Mentoring Workshop 2019 and 2020. I regularly participate in the department's REU program and have mentored 58 REU students from a wide range of different backgrounds since 2015, of which many published a paper and subsequently pursued a PhD (including three who joined my group).

[7] https://beyond-the-model-meeting.github.io

References

Apel, Sven, Christian Kästner, and Eunsuk Kang. 2022. “Feature Interactions on Steroids: On the Composition of ML Models.” *IEEE Software* 39 (5): 120–124.

Bastani, Hamsa, Osbert Bastani, Alp Sungu, Haosen Ge, Özge Kabakcı, and Rei Mariman. 2025. “Generative AI Without Guardrails Can Harm Learning: Evidence from High School Mathematics.” *Proceedings of the National Academy of Sciences* 122 (26). https://doi.org/10.1073/pnas.2422633122.

Bhat, Avinash, Austin Coursey, Grace Hu, et al. 2023. “Aspirations and Practice of Model Documentation: Moving the Needle with Nudging and Traceability.” *Proc. CHI*. http://arxiv.org/abs/2204.06425.

Biggs, John. 1996. “Enhancing Teaching through Constructive Alignment.” *Higher Education* 32 (3): 347–364.

Bjork, E., and R. Bjork. 2011. “Making Things Hard on Yourself, but in a Good Way: Creating Desirable Difficulties to Enhance Learning.” In *Psychology and the Real World*.

Black, Paul, and Dylan Wiliam. 1998. “Inside the Black Box: Raising Standards through Classroom Assessment.” *Phi Delta Kappan* 80 (2): 139–148.

Bogart, Christopher, Christian Kästner, James Herbsleb, and Ferdian Thung. 2016. “How to Break an API: Cost Negotiation and Community Values in Three Software Ecosystems.” *Proc. Int’l Symposium Foundations of Software Engineering (FSE)* (New York), 109–120.

Chenoweth, Steve, and Panagiotis K. Linos. 2025. “Teaching Machine Learning as Part of Agile Software Engineering.” *IEEE Transactions on Education* 68 (4): 322–335.

Costa, Diego. 2022. *SOEN 691: Engineering AI-Based Software Systems*. Github. https://github.com/create-se4ai/engineering-ai-systems-course.

Eghbal, Nadia. 2020. *Working in Public: The Making and Maintenance of Open Source Software*. Stripe Press.

Feldman, Joe. 2024. *Grading for Equity*. 2nd ed. Corwin Press.

Ferreira, Gabriel, Limin Jia, Joshua Sunshine, and Christian Kästner. 2021. “Containing Malicious Package Updates in Npm with a Lightweight Permission System.” *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, May, 1334–1346.

Handa, Kunal, Drew Bent, Alex Tamkin, et al. 2025. “Anthropic Education Report: How University Students Use Claude.” https://www.anthropic.com/news/anthropic-education-report-how-university-students-use-claude.

He, Hao, Bogdan Vasilescu, and Christian Kästner. 2025. “Pinning Is Futile: You Need More Than Local Dependency Versioning to Defend Against Supply Chain Attacks.” *Proceedings of the ACM on Software Engineering* 2 (FSE): 266–289.

He, Hao, Haoqin Yang, Philipp Burckhardt, Alexandros Kapravelos, Bogdan Vasilescu, and Christian Kästner. 2026. “Six Million (Suspected) Fake Stars on GitHub: A Growing Spiral of Popularity Contests, Spams, and Malware.” *Proceedings of the International Conference on Software Engineering*. https://arxiv.org/abs/2412.13459.

Hong, Yining, Yining She, Eunsuk Kang, Christopher Timperley, and Christian Kästner. 2026. *Symbolic Guardrails for Domain-Specific Agents: Stronger Safety and Security Guarantees Without Sacrificing Utility*. ArXiv. https://arxiv.org/abs/2604.15579.

Hong, Yining, Christopher Timperley, and Christian Kästner. 2025. “From Hazard Identification to Control Design: Proactive and AI-Supported Safety Engineering for ML-Powered Systems.” *Proceedings of the International Conference on AI Engineering - Software Engineering for AI (CAIN)*, 113–118.

Kästner, Christian. 2025. *Machine Learning in Production: From Models to Products*. MIT Press, open access.

Kästner, Christian, Sven Apel, and Martin Kuhlemann. 2008. “Granularity in Software Product Lines.” *Proc. Int’l Conf. Software Engineering (ICSE)*, 311–320.

Kästner, Christian, Paolo G. Giarrusso, Tillmann Rendel, Sebastian Erdweg, Klaus Ostermann, and Thorsten Berger. 2011. “Variability-Aware Parsing in the Presence of Lexical Macros and Conditional Compilation.” *Proc. Int’l Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)* (New York), 805–824.

Kästner, C., and E. Kang. 2020. “Teaching Software Engineering for Al-Enabled Systems.” *2020 IEEE/ACM 42nd International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*, October, 45–48.

Liebig, Jörg, Christian Kästner, and Sven Apel. 2011. “Analyzing the Discipline of Preprocessor Annotations in 30 Million Lines of C Code.” *AOSD* (New York), 191–202.

Lillack, Max, Christian Kästner, and Eric Bodden. 2017. “Tracking Load-Time Configuration Options.” *IEEE Transactions on Software Engineering*.

Lovett, Marsha C., Michael W. Bridges, Michele DiPietro, Susan A. Ambrose, and Marie K. Norman. 2023. *How Learning Works: Eight Research-Based Principles for Smart Teaching*. 2nd ed. Jossey-Bass.

Lyman, F. T. 1981. “The Responsive Classroom Discussion: The Inclusion of All Students.” *Mainstreaming Digest*.

Mayer, Roger C., James H. Davis, and F. David Schoorman. 1995. “An Integrative Model Of Organizational Trust.” *AMRO* 20 (3): 709–734.

Medeiros, Flávio, Christian Kästner, Márcio Ribeiro, Rohit Gheyi, and Sven Apel. 2016. “A Comparison of 10 Sampling Algorithms for Configurable Systems.” *Proc. Int’l Conf. Software Engineering (ICSE)*.

Meinicke, Jens, Chu-Pan Wong, Christian Kästner, Thomas Thüm, and Gunter Saake. 2016. "On Essential Configuration Complexity: Measuring Interactions In Highly-Configurable Systems." *Proc. Int'l Conf. Automated Software Engineering (ASE)* (New York), September, 483–494.

Miller, Courtney, Sophie Cohen, Daniel Klug, Bogdan Vasilescu, and Christian Kästner. 2022. "'Did You Miss My Comment or What?' Understanding Toxicity in Open Source Discussions." *Proceedings of the International Conference on Software Engineering*. https://doi.org/10.1145/3510003.3510111.

Miller, Courtney, Hao He, Weigen Chen, et al. 2026. "Designing Abandabot: When Does Open Source Dependency Abandonment Matter?" *Proceedings of the 48th International Conference on Software Engineering (ICSE)*.

Miller, Courtney, Mahmoud Jahanshahi, Audris Mockus, Bogdan Vasilescu, and Christian Kästner. 2025. "Understanding the Response to Open-Source Dependency Abandonment in the npm Ecosystem." *Proceedings of the International Conference on Software Engineering*.

Miller, Courtney, David Widder, Christian Kästner, and Bogdan Vasilescu. 2019. "Why Do People Give Up FLOSSIng? A Study of Contributor Disengagement in Open Source." *Proc. IFIP Int'l Conf. Open Source Systems (OSS)*, 116–129.

Nahar, Nadia, Christian Kästner, Jenna Butler, Chris Parnin, Thomas Zimmermann, and Christian Bird. 2025. "Beyond the Comfort Zone: Emerging Solutions to Overcome Challenges in Integrating LLMs into Software Products." *Proceedings of the Proc. International Conference on Software Engineering -- Software Engineering in Practice Track (ICSE-SEIP)*, 516–527.

Nahar, Nadia, Zahra Abba Omar, Jacob Tjaden, et al. 2026. "Policy Alone Is Probably Not the Solution: A Large-Scale Experiment on How Developers Struggle to Design Meaningful End-User Explanations." In *arXiv*. https://doi.org/10.48550/arXiv.2503.15512.

Nahar, Nadia, Jenny Rowlett, Matthew Bray, et al. 2024. "Regulating Explainability in Machine Learning Applications -- Observations from a Policy Design Experiment." *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2101–2112.

Nahar, Nadia, Chenyang Yang, Yanxin Chen, et al. 2026. "'I Don't Think RAI Applies to My Model' -- Engaging Non-Champions with Sticky Stories for Responsible AI Work." *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems (CHI)*.

Nahar, Nadia, Haoran Zhang, Grace Lewis, Shurui Zhou, and Christian Kästner. 2023. "A Meta-Summary of Challenges in Building Products with ML Components – Collecting Experiences from 4758+ Practitioners." *Proceedings of the International Conference on AI Engineering - Software Engineering for AI (CAIN)*.

Nahar, Nadia, Shurui Zhou, Grace Lewis, and Christian Kästner. 2022. "Collaboration Challenges in Building ML-Enabled Systems." *Proceedings of the 44th International Conference on Software Engineering*. https://doi.org/10.1145/3510003.3510209.

Nilson, Linda B. 2015. *Specifications Grading: Restoring Rigor, Motivating Students, and Saving Faculty Time*. Stylus Publishing, LLC.

Oakley, Barbara, Richard M. Felder, Rebecca Brent, and Imad Elhajj. 2004. “Turning Student Groups into Effective Teams.” *Journal of Student Centered Learning* 2 (1): 9–34.

Ostermann, Klaus, Paolo G. Giarrusso, Christian Kästner, and Tillmann Rendel. 2011. “Revisiting Information Hiding: Reflections on Classical and Nonclassical Modularity.” *Proc. Europ. Conf. Object-Oriented Programming (ECOOP)*, 155–178.

Palomba, Fabio, Gianmario Voria, Alessandra Parziale, et al. 2026. “Teaching Software Engineering for Artificial Intelligence: An Experience Report.” *Proceedings of Software Engineering and Advanced Applications*. https://doi.org/10.1007/978-3-032-04207-1_15.

Patton, Bruce, Douglas Stone, and Sheila Heen. 2021. *Difficult Conversations*. Penguin Books.

Putnam, R. D. 2015. “‘bowling Alone: America’s Declining Social Capital.’” In *The City Reader*. Routledge.

Rhein, Alexander Von, Jörg Liebig, Andreas Janker, Christian Kästner, and Sven Apel. 2018. “Variability-Aware Static Analysis at Scale: An Empirical Study.” *ACM Transactions on Software Engineering and Methodology* 27 (4): 18:1–18:33.

Siegmund, Norbert. 2025. “10-202-2345 Software Engineering für KI-Systeme.” https://almaweb.uni-leipzig.de/scripts/mgrqispi.dll?APPNAME=CampusNet&PRGNAME=MODULEDETAILS&ARGUMENTS=-N000000000000001%2C-N000573%2C-N395054599113630%2C-A.

Stein, Ruth Federman, and Sandra N. Hurd. 2000. *Using Student Teams in the Classroom: A Faculty Guide*. Anker Publishing Company.

Veinott, Beth, G. Klein, and S. Wiggins. 2010. “Evaluating the Effectiveness of the PreMortem Technique on Plan Confidence.” *Proceedings of the 7th International ISCRAM Conference*.

Velez, Miguel, Pooyan Jamshidi, Norbert Siegmund, Sven Apel, and Christian Kästner. 2022. “On Debugging the Performance of Configurable Software Systems: Developer Needs and Tailored Tool Support.” *Proceedings of the 44th International Conference on Software Engineering*, ICSE ’22, 1571–1583.

Yang, Chenyang, Rachel A. Brower-Sinning, Grace Lewis, and Christian Kästner. 2023. “Data Leakage in Notebooks: Static Detection and Better Processes.” *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, Article 30.

Yang, Chenyang, Rishabh Rustogi, Rachel Brower-Sinning, Grace A. Lewis, Christian Kästner, and Tongshuang Wu. 2023. “Beyond Testers’ Biases: Guiding Model Testing with Knowledge Bases Using LLMs.” *Proceedings of the Conference on Empirical Methods in Natural Language Processing -- Findings (EMNLP)*.

Yang, Chenyang, Yike Shi, Qianou Ma, Michael Xieyang Liu, Christian Kästner, and Tongshuang Wu. 2026. “What Prompts Don’t Say: Understanding and Managing Underspecification in LLM Prompts.” *Proceedings of the Annual Meeting of the Association for Computational Linguistics -- Findings (ACL)*.

Yang, Chenyang, Shurui Zhou, Jin L. C. Guo, and Christian Kästner. 2021. “Subtle Bugs Everywhere: Generating Documentation for Data Wrangling Code.” *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Zhou, Shurui, Bogdan Vasilescu, and Christian Kästner. 2019. “What the Fork: A Study of Inefficient and Efficient Forking Practices in Social Coding.” *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 350–361.

Zhou, S., S. Stanciulescu, O. Leßenich, Y. Xiong, A. Wasowski, and C. Kästner. 2018. “Identifying Features in Forks.” *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*, May, 105–116.